

```
import os
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
dataset_path = "/content/drive/MyDrive/ANNlab/Pistachio_Image_Dataset/Pistachio_Image_Dataset"
```

```
# Define image transformations
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.125])
])
```

```
class PistachioDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.images = []
        self.labels = []

        #Load images from "Kirmizi_Pistachio" folder (Class 0)
        kirmizi_folder = os.path.join(root_dir, "Kirmizi_Pistachio")
        for file in os.listdir(kirmizi_folder):
            if file.lower().endswith(('jpg', 'png', 'jpeg')):
                self.images.append(os.path.join(kirmizi_folder, file))
                self.labels.append(0) # 0 for Kirmizi_Pistachio

        # Load images from "Siirt_Pistachio" folder (Class 1)
        siirt_folder = os.path.join(root_dir, "Siirt_Pistachio")
        for file in os.listdir(siirt_folder):
            if file.lower().endswith(('jpg', 'png', 'jpeg')):
                self.images.append(os.path.join(siirt_folder, file))
                self.labels.append(1) # 1 for Siirt_Pistachio

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        # Load image and label
        img_path = self.images[idx]
        label = self.labels[idx]

        # Open image
        image = Image.open(img_path).convert("L")

        # Apply transformations
        if self.transform:
            image = self.transform(image)

        # Convert label to tensor
        label = torch.tensor(label, dtype=torch.long)

        return image, label
```

```
# Instantiate dataset
dataset = PistachioDataset(dataset_path, transform=transform)
```

```
# Create DataLoader
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)
```

```
# Test the DataLoader
for images, labels in dataloader:
    print(f"Batch size: {images.shape}, Labels: {labels}")
    break
```

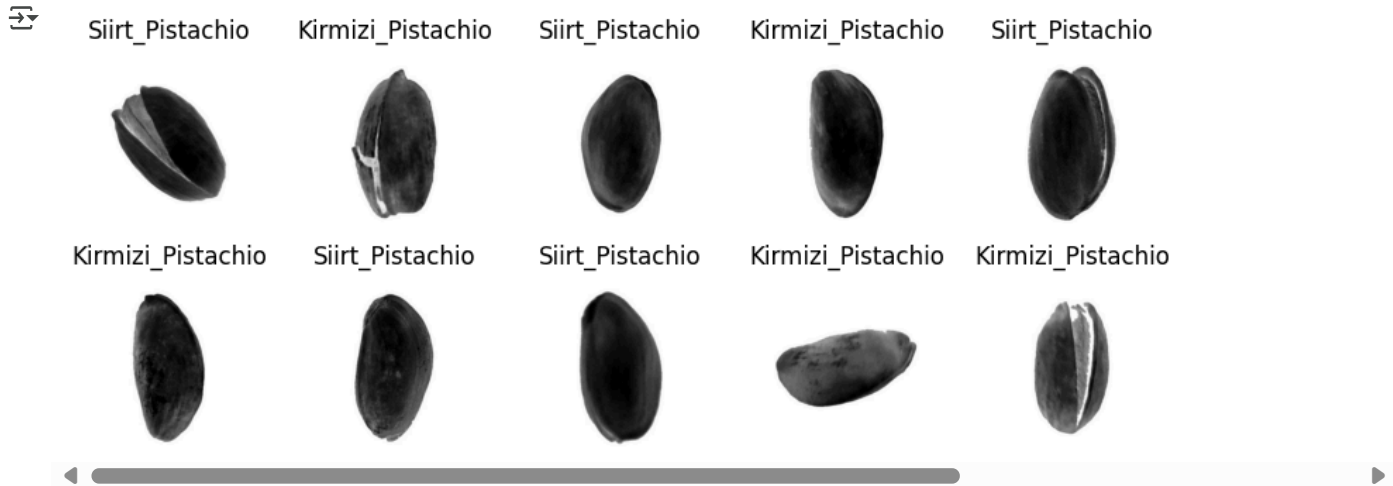
Batch size: torch.Size([64, 1, 256, 256]), Labels: tensor([1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1])

```

image, labels = next(iter(dataloader))
class_name = ['Kirmizi_Pistachio', 'Siirt_Pistachio']
plt.figure(figsize = (10,10))
for i in range(10):
    plt.subplot(5,5,i+1)
    img = image[i].permute(1,2,0)
    img = img *0.123 + 0.5
    plt.imshow(img.numpy(), cmap = 'Grays')
    plt.title(f"{class_name[labels[i].item()]}")
    plt.axis("off")

plt.show()

```



```

dataset = PistachioDataset(dataset_path, transform=transform)

train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size
train_dataset, test_dataset = torch.utils.data.random_split(dataset, [train_size, test_size])

dataloader_train = DataLoader(train_dataset, batch_size=64, shuffle=True)
dataloader_test = DataLoader(test_dataset, batch_size=64, shuffle=False)

x_train, y_train = [], []
for images, labels in dataloader_train:
    x_train.append(images.numpy().reshape(images.shape[0], -1)) # Flatten images
    y_train.append(labels.numpy())

x_test, y_test = [], []
for images, labels in dataloader_test:
    x_test.append(images.numpy().reshape(images.shape[0], -1))
    y_test.append(labels.numpy())

import numpy as np
x_train, y_train = np.vstack(x_train), np.hstack(y_train)
x_test, y_test = np.vstack(x_test), np.hstack(y_test)

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from torch.utils.data import Dataset, DataLoader

model = keras.Sequential([
    layers.Dense(512, activation='relu', input_shape=(256*256,)), # Input layer
    layers.Dense(256, activation='relu'),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid') # Output layer (binary classification)
])

```

```
# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape` to
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

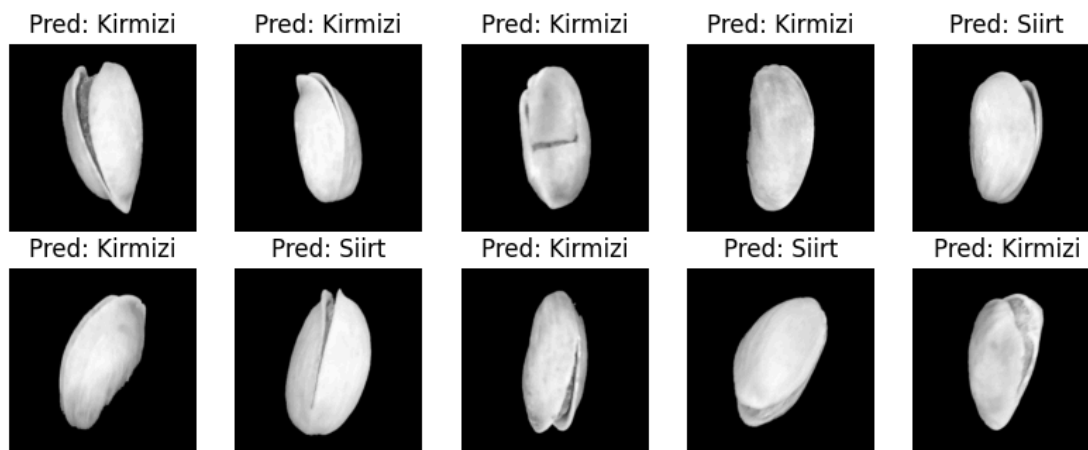
```
history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_data=(x_test, y_test))
```

```
Epoch 1/10
27/27 ----- 23s 828ms/step - accuracy: 0.8394 - loss: 0.8779 - val_accuracy: 0.8233 - val_loss: 0.8326
Epoch 2/10
27/27 ----- 41s 850ms/step - accuracy: 0.7719 - loss: 1.2717 - val_accuracy: 0.8814 - val_loss: 0.5306
Epoch 3/10
27/27 ----- 21s 797ms/step - accuracy: 0.8160 - loss: 1.4151 - val_accuracy: 0.8860 - val_loss: 0.4435
Epoch 4/10
27/27 ----- 40s 771ms/step - accuracy: 0.7636 - loss: 1.7842 - val_accuracy: 0.6558 - val_loss: 6.0718
Epoch 5/10
27/27 ----- 42s 792ms/step - accuracy: 0.7900 - loss: 3.0985 - val_accuracy: 0.8698 - val_loss: 0.4509
Epoch 6/10
27/27 ----- 23s 854ms/step - accuracy: 0.8528 - loss: 0.5452 - val_accuracy: 0.8791 - val_loss: 0.3561
Epoch 7/10
27/27 ----- 49s 1s/step - accuracy: 0.8201 - loss: 0.8207 - val_accuracy: 0.8767 - val_loss: 0.3846
Epoch 8/10
27/27 ----- 33s 833ms/step - accuracy: 0.8008 - loss: 0.9662 - val_accuracy: 0.8395 - val_loss: 0.8157
Epoch 9/10
27/27 ----- 40s 804ms/step - accuracy: 0.8418 - loss: 0.6708 - val_accuracy: 0.8837 - val_loss: 0.3022
Epoch 10/10
27/27 ----- 41s 819ms/step - accuracy: 0.8500 - loss: 0.3928 - val_accuracy: 0.8860 - val_loss: 0.3231
```

```
predictions = model.predict(x_test[:10])
predicted_labels = (predictions > 0.5).astype(int)
```

```
plt.figure(figsize=(10, 10))
for i in range(10):
    plt.subplot(5, 5, i + 1)
    img = x_test[i].reshape(256, 256)
    plt.imshow(img, cmap='gray')
    plt.title(f"Pred: {'Kirmizi' if predicted_labels[i] == 0 else 'Siirt'}")
    plt.axis("off")
plt.show()
```

```
1/1 ----- 0s 464ms/step
```



Start coding or [generate](#) with AI.



